ThreatMatrix AI — E2E Real Traffic Walkthrough Plan (Task 3)

> **Reference:** MASTER_DOC_PART5 §8.1 Demo Script (5:30-10:30 mark) **Date:** 2026-04-03 **VPS:** 187.124.45.161:8000 **Frontend:** http://localhost:3000 (local dev server) **Mode:** Live E2E testing with fresh alerts, full documentation **Execution:** Manual VPS commands + browser testing + API verification

---

Phase 0: VPS Preparation — Start ML Worker and Capture Engine

Purpose: Ensure all VPS components are running for live E2E testing

Commands to run on VPS (SSH into 187.124.45.161):

```
# 1. Check current container status
cd /path/to/threatmatrix-ai  # Navigate to project directory
docker-compose ps

# 2. Restart ML Worker for live scoring
docker-compose restart ml-worker

# 3. Restart Capture Engine for live traffic
docker-compose restart capture

# 4. Restart backend to ensure clean state
docker-compose restart backend

# 5. Verify all containers running
docker-compose ps
# Expected: All 5 containers show "Up" status

# 6. Check health endpoint
curl -s http://localhost:8000/api/v1/system/health | python3 -m json.tool
# Expected: All components show "healthy" or "active"

# 7. Verify ML models loaded
curl -s http://localhost:8000/api/v1/ml/models | python3 -m json.tool
# Expected: 3 models listed (isolation_forest, random_forest, autoencoder)

# 8. Verify capture status
curl -s http://localhost:8000/api/v1/capture/status | python3 -m json.tool
# Expected: Status shows "running" or "active"
```

Expected Health Output:

```
{
  "status": "operational",
```

```
    "components": {
      "api": "healthy",
      "database": "healthy",
      "redis": {"status": "healthy", "latency_ms": <5},
      "capture_engine": "active",
      "ml_worker": "active"
    }
  }
```

Pass Criteria: All 5 containers running, health endpoint shows all components healthy/active.

Documentation Required:

- Output of `docker-compose ps`
- Health endpoint response
- ML models response
- Capture status response

Pre-Flight Checklist

Before starting the walkthrough, verify:

Check	Command/URL	Expected	Status
Backend API	`curl http://187.124.45.161:8000/api/v1/system/health`	status: operational	⚠ Verified (DB pending, ML idle)
Frontend	http://localhost:3000	Page loads	☐ Start with `npm run dev`
WebSocket	Browser console → check WS connection	Connected	☐
Attack scripts ready	`scripts/attack_simulation/run_external_attacks.py`	File exists	☑
PCAP files ready	`pcaps/demo/*.pcap`	5 files exist	☑
nmap installed	`nmap --version`	Available	☐

Step-by-Step Walkthrough

Step 1: War Room Live Data Verification

Route: `/war-room` **Demo Script Reference:** "Open War Room. Live data flowing. Map active."

Pre-requisite: Start frontend with `cd frontend && npm run dev`

Action	Expected Result	Pass/Fail	Notes
Open http://localhost:3000/war-room	Page loads without errors	☐	
Check threat level gauge	Shows current threat level	☐	
Verify live metrics cards	Packets/s, flows, anomaly% display	☐	
Check threat map	Deck.gl map renders with data points	☐	
Verify live alert feed	Shows recent alerts (if any)	☐	
Check protocol distribution	Pie chart displays	☐	
Check traffic timeline	Area chart shows historical data	☐	
Open browser DevTools → Console	No errors	☐	
Open browser DevTools → Network	API calls succeed (200)	☐	

API Endpoints Called:

- `GET /api/v1/flows/stats` — aggregated flow statistics
- `GET /api/v1/alerts/` — recent alerts
- `GET /api/v1/flows/protocols` — protocol distribution
- `GET /api/v1/flows/top-talkers` — top IPs

Pass Criteria: Page loads, displays data from VPS, no console errors.

Documentation Required:

- Screenshot of War Room with live data
- API response times for each endpoint
- Console errors (if any)

Step 2: Attack → Alert Detection (60s Target)

Demo Script Reference: "Run nmap attack → alert fires"

Approach: Live external attack from local machine → VPS public IP

Action	Expected Result	Pass/Fail	Notes
Record T0 (before attack)	T0 = _____	☐	Format: HH:MM:SS
Run port scan from local machine	`nmap -sS -p 1-1024 187.124.45.161`	☐	
Monitor War Room live alert feed	New alert appears	☐	

Action	Expected Result	Pass/Fail	Notes
Record T1 (alert appearance)	T1 = _____	☐	Format: HH:MM:SS
Calculate latency	T1 - T0 = _____ seconds	☐	Target: <60s
Verify alert category	category = "port_scan"	☐	
Verify alert severity	severity ≥ MEDIUM	☐	
Verify ML confidence	confidence ≥ 50%	☐	
Verify LLM narrative generated	ai_narrative field populated	☐	

Attack Command (from local Windows machine):

```
# Using nmap (install if needed: https://nmap.org/download.html)
nmap -sS -p 1-1024 187.124.45.161

# Alternative: Using Python script (cross-platform)
python scripts/attack_simulation/run_external_attacks.py --target 187.124.45.161 -
-attack port_scan
```

Verification Commands (run on VPS after attack):

```
# Check latest alerts
curl -s http://187.124.45.161:8000/api/v1/alerts/?limit=5 | python3 -m json.tool

# Check flow count increased
curl -s http://187.124.45.161:8000/api/v1/flows/stats | python3 -m json.tool
```

Pass Criteria: Alert appears within 60 seconds, correctly classified as port_scan, LLM narrative generated.

Documentation Required:

- T0 and T1 timestamps
- Full API response for the new alert
- Screenshot of alert appearing in War Room feed

Step 3: Alert Console with AI Narrative

Route: `/alerts` **Demo Script Reference:** "Alert fires → AI explains"

Action	Expected Result	Pass/Fail	Notes
Navigate to http://localhost:3000/alerts	Page loads with alert list	☐	

Action	Expected Result	Pass/Fail	Notes
Find alert from Step 2	Alert visible in list (should be recent)	☐	Note alert ID
Click alert to open detail	Detail drawer/modal opens	☐	
Check AI narrative field	Text is present (not null/empty)	☐	
Read AI narrative	Explains port scan activity	☐	Copy narrative text
Verify narrative quality	Clear, professional language	☐	Rate 1-5
Check related flows	Flow data displayed	☐	
Check IOC matches	Any IOC correlation shown	☐	
Check alert metadata	severity, confidence, category correct	☐	

API Endpoints Called:

- `GET /api/v1/alerts/` — list alerts
- `GET /api/v1/alerts/{id}` — alert detail with AI narrative
- `GET /api/v1/flows/{id}` — related flow data

Pass Criteria: Alert detail loads, AI narrative is present and explains the attack.

Documentation Required:

- Alert ID from Step 2
- Full AI narrative text (copy verbatim)
- Quality rating (1-5 scale)
- Screenshot of alert detail page

Step 4: AI Analyst Coherent Response

Route: `/ai-analyst` **Demo Script Reference:** "Ask AI about the attack. Generate briefing."

Action	Expected Result	Pass/Fail	Notes
Navigate to http://localhost:3000/ai-analyst	Chat interface loads	☐	
Check context selector	Shows "General" or alert context	☐	
Send query: "Explain the latest port scan alert"	Message sent	☐	Record T2
Observe streaming response	Tokens stream in (typing effect)	☐	

Action	Expected Result	Pass/Fail	Notes
Record response complete time	_____ seconds	☐	Record T3, latency = T3-T2
Check response relevance	Mentions nmap, SYN scan, ports	☐	Rate 1-5
Check technical accuracy	No hallucinations or errors	☐	
Test quick action: "Analyze Latest Alert"	Works correctly	☐	
Check token budget display	Shows remaining budget	☐	
Copy full AI response	For documentation	☐	

API Endpoints Called:

- `POST /api/v1/llm/chat` — streaming chat (SSE)
- `POST /api/v1/llm/analyze-alert/{id}` — alert analysis (if used)

Pass Criteria: AI responds within 10s, response is technically accurate and relevant.

Documentation Required:

- Full query and response text
- Response latency (T3 - T2)
- Quality rating (1-5 scale)
- Screenshot of chat interface

Step 5: ML Ops Metrics Display

Route: `/ml-ops` **Demo Script Reference:** "Model comparison table. Confusion matrices. ROC curves."

Action	Expected Result	Pass/Fail	Notes
Navigate to http://localhost:3000/ml-ops	Page loads	☐	
Check model registry	3 models listed (IF, RF, AE)	☐	Note model IDs
Verify model comparison table	Metrics displayed for all 3	☐	
Check confusion matrix	Heatmap renders	☐	
Check ROC curves	Chart displays curves	☐	
Check feature importance	Bar chart shows top features	☐	
Verify accuracy numbers	IF: ~82%, RF: ~74%, AE: ~62%	☐	Per handoff doc
Check ensemble score	~80% accuracy	☐	
Check training history	Historical runs logged	☐	

API Endpoints Called:

- `GET /api/v1/ml/models` — list all models
- `GET /api/v1/ml/comparison` — model comparison
- `GET /api/v1/ml/confusion-matrix` — confusion matrix data
- `GET /api/v1/ml/feature-importance` — feature importance
- `GET /api/v1/ml/training-history` — training runs

Pass Criteria: All visualization components load with real model data.

Documentation Required:

- Model metrics table (copy values)
- Screenshot of ML Ops dashboard
- Any console errors or rendering issues

Step 6: Reports PDF Generation

Route: `/reports` **Demo Script Reference:** "Report generation (PDF)"

Action	Expected Result	Pass/Fail	Notes
Navigate to http://localhost:3000/reports	Page loads	☐	
Click "Generate Daily Threat Summary"	Generation starts	☐	Record T4
Wait for completion	Report appears in list	☐	Record T5, latency = T5-T4
Download PDF	File downloads	☐	Note filename
Open PDF	ThreatMatrix branding visible	☐	
Verify content	Alert data, AI narrative included	☐	
Check formatting	Professional layout	☐	Rate 1-5
Verify alert from Step 2 is in report	Alert ID present	☐	

API Endpoints Called:

- `POST /api/v1/reports/generate` — trigger generation
- `GET /api/v1/reports/` — list reports
- `GET /api/v1/reports/{id}/download` — download PDF

Pass Criteria: PDF generates successfully with correct data and branding.

Documentation Required:

- Generation latency (T5 - T4)
- PDF filename and size

- Screenshot of reports page
- Quality rating (1-5 scale)

Step 7: Intel Hub IOC Data

Route: `/intel` **Demo Script Reference:** "Cross-reference with threat intel"

Action	Expected Result	Pass/Fail	Notes
Navigate to http://localhost:3000/intel	Page loads	☐	
Check IOC browser	IOCs displayed in table	☐	Note count
Verify feed status	OTX, AbuseIPDB, VT status shown	☐	
Check IOC count	Count > 0 (1,367 per handoff)	☐	
Test IP lookup	Enter 8.8.8.8, get results	☐	Record response
Check correlation panel	IOCs matched to internal flows	☐	

API Endpoints Called:

- `GET /api/v1/intel/iocs` — list IOCs
- `GET /api/v1/intel/lookup/{ip}` — IP reputation
- `GET /api/v1/intel/feeds/status` — feed health

Pass Criteria: IOC data displays, lookup returns results.

Documentation Required:

- IOC count displayed
- Feed status for each provider
- IP lookup response
- Screenshot of Intel Hub

Step 8: Admin Audit Log

Route: `/admin` or `/admin/audit` **Demo Script Reference:** N/A (enterprise feature verification)

Action	Expected Result	Pass/Fail	Notes
Navigate to http://localhost:3000/admin	Admin page loads	☐	
Navigate to audit log section	Events displayed	☐	
Check audit log entries	Events displayed with details	☐	Note count
Verify event types	login, alert_create, etc.	☐	List types seen

Action	Expected Result	Pass/Fail	Notes
Check timestamps	Recent events visible	☐	
Verify user attribution	Events linked to users	☐	

API Endpoints Called:

- `GET /api/v1/admin/audit-log` — audit log entries

Pass Criteria: Audit log displays events with timestamps and user attribution.

Documentation Required:

- Event count displayed
- Event types observed
- Most recent event details
- Screenshot of audit log

Summary Results Template

Step	Component	Pass/Fail	Latency	Quality (1-5)	Notes
1	War Room	☐	—	—	
2	Attack → Alert	☐	_____ s	—	
3	Alert Console	☐	—	___	
4	AI Analyst	☐	_____ s	___	
5	ML Ops	☐	—	—	
6	Reports PDF	☐	_____ s	___	
7	Intel Hub	☐	—	—	
8	Admin Audit	☐	—	—	

Overall Pass Rate: ***/8 (***%) **Average Quality Score:** ___/5

API Endpoint Verification Log

Endpoint	Method	Status Code	Response Time	Response Size	Notes
/api/v1/system/health	GET				
/api/v1/flows/stats	GET				
/api/v1/alerts/	GET				
/api/v1/alerts/{id}	GET				
/api/v1/flows/protocols	GET				

Endpoint	Method	Status Code	Response Time	Response Size	Notes
/api/v1/flows/top-talkers	GET				
/api/v1/ml/models	GET				
/api/v1/ml/comparison	GET				
/api/v1/ml/confusion-matrix	GET				
/api/v1/ml/feature-importance	GET				
/api/v1/llm/chat	POST				
/api/v1/reports/generate	POST				
/api/v1/reports/	GET				
/api/v1/intel/iocs	GET				
/api/v1/intel/lookup/{ip}	GET				
/api/v1/intel/feeds/status	GET				
/api/v1/admin/audit-log	GET				

Known Issues to Watch

1. **Database "pending" status** — Should resolve after Phase 0 restart
2. **ML Worker "idle"** — Should resolve after Phase 0 restart
3. **Capture Engine "idle"** — Should resolve after Phase 0 restart
4. **DEV_MODE** — Auth may be bypassed; verify if needed
5. **nmap on Windows** — May need installation; alternative is Python script

Recommended Execution Order

1. **Phase 0:** VPS preparation — restart containers, verify health
2. **Phase 1:** Start frontend locally, verify War Room
3. **Phase 2:** Run live attack (nmap or Python script), verify alert detection
4. **Phases 3-8:** Proceed sequentially through remaining steps
5. **Phase 9:** Frontend component audit and polish
6. **Phase 10:** Write E2E walkthrough report

Fallback Options

Risk	Fallback
External attack doesn't produce alert	Use PCAP upload instead
AI Analyst times out	Check pre-existing alert narratives
PDF generation fails	Check backend logs, verify ReportLab installed

Risk	Fallback
Frontend has errors	Check browser console, verify API connectivity
VPS components idle	Restart containers: `docker-compose restart`

Day 19 Task 3 — E2E Real Traffic Walkthrough Plan Reference: PART5 §8.1 Demo Script, SESSION_HANDOFF.md